

COMP 3106 – Assignment 3

Date: November 5th, 2025

Group 133 Members:

Abaan Noman (101305538),
Emily Amos (101311817),
Rahul Rodrigues (101145082)

Statement of Contributions:

All group members made approximately equal contributions.

Team:

- Brainstorming ideas to implement temporal difference q-learning
- PDF report and questions

Abaan:

- Implemented state parsing, terminal state checking, and reward function
- Helped test Q-value correctness against provided Example trials

Emily:

- Discussed Q-learning update rule and convergence stopping criteria
- Reviewed policy tie-breaking and testing strategy

Rahul:

- Implemented Q-learning update loop, target computation, and lazy initialization
- Created and ran the automated test script for evaluating Examples 0–3

- 1) What type of agent have you implemented (simple reflex agent, model-based reflex agent, goal-based agent, or utility-based agent)?

The agent implemented in this assignment is a utility-based agent.

The agent does not choose actions based on condition-action rules (so it is not a simple reflex agent). It also does not maintain or use an explicit model of how the environment transitions between states (so it is not a model-based reflex agent). Also, the agent is not explicitly given a goal state to search toward, so it is not a goal-based agent.

Instead, the agent learns a numerical utility (Q-value) for each state-action pair based on the reward signal defined in the environment. Using Q-learning, the agent estimates how good each action is in each state and selects the action that maximizes expected return. This matches the definition of a utility-based agent, which evaluates possible actions in terms of expected utility and acts to maximize it.

Therefore, the implementation corresponds to a utility-based agent.

- 2) The task environment is:

- a) Fully observable

The state string $c_{bag}/c_{agent}/c_{opponent}/winner$ fully specifies all relevant information in the environment. There is no hidden state.

- b) Multi-agent

The game involves our agent and an opponent, who alternates turns.

- c) Deterministic

The result of taking an action (removing 1-3 coins) always leads to a uniquely determined next state. There is no randomness in transitions or outcomes.

- d) Sequential

Each action affects future states and rewards. The agent's decisions matter across a sequence of turns.

- e) Static

The environment does not change while the agent is deciding; time does not alter the environment except through agent/opponent actions.

- f) Discrete

States are represented by integer counts of coins and winner values, and actions are discrete choices (1, 2, 3).

- g) Unknown

The agent is not given a transition model or reward model ahead of time. Instead, the agent must learn the Q-values from experience (aka offline trials).

- 3) For some cases (even with many trials through state space), the optimal Q-value for a particular state-action pair will not be found. Explain why this might be the case.

Since the trials were generated using a random or fixed policy, some state-action pairs may occur only rarely or not at all. Q-learning is an off-policy learning method, but it still relies on data to update estimates. If a pair does not appear in the experiment, the agent has no evidence to improve its estimate. Therefore the optimal Q-value is not learned.

- 4) In the test cases provided, the trials through state space were simulated using a random policy. Describe a different strategy to simulate trials and compare it to using a random policy.

A different strategy is to generate trials using a greedy policy (G) based on the current Q-values. Under G-greedy, the agent selects the best-known action with probability $1 - G$, and a random action with probability G . As learning progresses, G can be decreased to shift from exploration to exploitation.

Random Policy	Greedy policy
Chooses all actions uniformly at random.	Chooses mostly high-value actions, but still explores.
Provides good exploration, but many trajectories will be poor and uninformative.	Produces more reward-relevant experience while still exploring unseen states.
Learning may be slow because useful state-action outcomes are seen infrequently.	Learning is typically faster and more stable, since it focuses experience on promising strategies.

While a random policy ensures broad coverage, a greedy policy balances exploration and exploitation. This allows Q-values to converge more efficiently to strong strategies.

- 5) Suppose we use a large state space approximation for learning Q-values for this task.

$$Q(s, a) = \theta_1 f_1(s, a) + \dots + \theta_n f_n(s, a)$$

Let the state be $(c_{bag}, c_{agent}, c_{opponent}, winner)$ and the action $a \in \{1, 2, 3\}$.

We can define the following basis (features) functions:

$$f_1(s, a) = c_{bag}$$

$$f_2(s, a) = c_{agent}$$

$$f_3(s, a) = c_{opponent}$$

$$f_4(s, a) = 1 [winner = A]$$

$$f_5(s, a) = 1 [winner = O]$$

$$f_6(s, a) = 1 [a = 1], f_7(s, a) = 1 [a = 2], f_8(s, a) = 1 [a = 3]$$

- f_1 captures how close the game is to ending; the value of actions depends heavily on remaining coins.
- f_2 and f_3 capture the number of coins held by the agent and opponent, which determines the final reward at terminal states.
- f_4 and f_5 allow the function to represent large positive or negative terminal rewards.
- f_6, f_7 , and f_8 distinguish between the three possible actions, enabling the model to learn different utilities for taking 1, 2, or 3 coins.

These features help represent the key parts of the game state (remaining coins and current scores) and also distinguish between the possible actions. This lets the agent learn useful patterns and make good decisions, even without storing a separate Q-value for every possible state.

- 6) An alternative approach to find the optimal strategy for this two-player adversarial game is to use minimax search. Suppose your opponent's reward function is analogous to your agent's.

$$r_{opponent}(s) = \begin{cases} c_{opponent} & \text{if } winner == A \\ -c_{opponent} & \text{if } winner == O \\ 0 & \text{otherwise} \end{cases}$$

To use minimax search in this environment, no changes to the game rules or state representation are required, since the game already alternates control between two players and the state representation $c_{bag}/c_{agent}/c_{opponent}/winner$ fully captures whose turn it is and whether the game is terminal.

The modifications needed are only in the evaluation function and the turn handling:

- a) Alternate between max and min layers:

- When it is our agent's turn, choose the action that maximizes the evaluation value.
- When it is the opponent's turn, choose the action that minimizes the evaluation value (assuming the opponent acts optimally).

- b) Use the correct utility function:

We evaluate terminal states using the reward definitions already provided:

$$r_{agent}(s) = \begin{cases} c_{agent} & \text{if } winner = A \\ -c_{agent} & \text{if } winner = O \\ 0 & \text{otherwise} \end{cases}$$

And for opponent turns, the opponent's reward is the negative of the agent's, matching the existing symmetric reward structure.

So we do not need to introduce a new reward function, minimax simply treats the opponent as a minimizing player.

c) Evaluation of non-terminal states:

For non-terminal states at search depth limits, the evaluation function should estimate value using:

- Difference in coins collected, OR
- Heuristic estimates of who is closer to forcing the opponent to take the last coin.

7) Under what conditions will your methods using reinforcement learning yield better rewards than minimax search? Under what conditions will minimax search yield better rewards than reinforcement learning? Provide justification.

Reinforcement learning performs better when:

- The opponent does not play optimally.
Q-learning learns from the actual behavior of the opponent in the trials. If the opponent makes mistakes or follows a predictable non-optimal strategy, the learned Q-values adapt to exploit those mistakes and can achieve higher rewards than a strategy that assumes perfect play.
- The game tree is too large to search fully and looking at every possible branch is too computationally expensive. RL does not require exploring the entire state space and can learn good strategies from experience alone.

Minimax performs better when:

- The opponent plays optimally.
Minimax explicitly assumes a rational opponent and computes a strategy that is guaranteed optimal under perfect play. If both players play rationally, minimax finds the provably optimal conservative policy, while RL may converge to a policy that performed well against weaker play but is not optimal in worst-case situations.
- The transition dynamics and rewards are known and exact search is feasible. If the entire game tree can be searched (or pruned effectively), minimax gives the true optimal value, whereas RL only approximates values based on sampled experience.

Summary:

Method	Performs Best When	Reason
Reinforcement Learning	Opponent plays non-optimally or game tree is large.	RL adapts to observed behavior and exploits mistakes. It does not require full search.
Minimax Search	Opponent plays optimally and game dynamics are known.	Minimax computes a provably optimal policy in perfect-play settings.